PACT

File-by-file implementation blueprint | 23 September 2026 | Version 1.0

This document tells a coding agent exactly which files to create and what each must implement. It is a blueprint, not existing source code. Pair it with Pact Technical Architecture and Build Spec for protocol rationale, economic invariants and lifecycle behavior. Build one functional vertical slice before broadening adapters or UI.

Target: Monad EVM contracts, TypeScript API and frontend, Python SDK, shared schemas, MCP server, PostgreSQL projections, encrypted evidence storage and a controlled external API gateway. Pin exact dependencies after checking current official package documentation.

1. Monorepo and operating assumptions

```
pact/
  apps/{api,gateway,mcp,console,explorer,indexer}/
  packages/{contracts,schemas,engine,sdk-ts,sdk-python,proofs}/
  examples/procurement-agent/
  infra/{docker,deploy,monitoring}/
  tests/{integration,e2e,invariants}/
  docs/{protocol,api,operations}/
```

Decision	Default for implementation
Package layout	pnpm workspace and TypeScript strict mode; Python package in its own project; Solidity Foundry project.
Assets	One deployed stablecoin in demo environment; amount in token base units; explicit token address.
Custody	Principal funds PactAccount and SettlementVault. No unrestricted agent signer or provider secret.
Valuation	Single asset for MVP. Do not pretend to normalize multiple tokens without a documented oracle.
Execution	Known typed adapters only. Unknown contract calls and unbounded external API calls deny.
Visibility	Private policy and evidence offchain encrypted; hashes and public commitments on Monad.

Cross-package contract

```
schemas -> contracts + engine + SDKs + services + apps
contracts -> chain events -> indexer -> read API -> Console/Explorer
engine -> decision/quote -> approval -> chain reservation/execution
SDK/MCP -> API -> engine; Gateway -> engine + vault + provider
```

2. Root files and shared configuration

package.json / pnpm-workspace.yaml

Implement: Define scripts for lint, typecheck, test, contract test, local stack and builds. Include only actual workspace packages. Interface: pnpm dev, pnpm test, pnpm check. Depends on: Selected package manager versions. Done when: A clean checkout installs and runs CI tasks.

tsconfig.base.json / .eslintrc / .prettierrc

Implement: Strict TypeScript defaults; no implicit any; consistent paths and build output. Interface: Shared compiler and lint settings. Depends on: Root workspace. Done when: Every TypeScript package typechecks.

.env.example / config/networks.ts

Implement: Document RPC, chain ID, deployed addresses, vault provider, database, object storage and auth settings; never include real secrets. Interface: Validated typed environment loader. Depends on: Monad environment confirmation. Done when: Startup fails with a clear error for missing or inconsistent values.

README.md / CONTRIBUTING.md / SECURITY.md

Implement: Repository purpose, local start, testnets, threat boundaries, responsible vulnerability reporting and disclosure policy. Interface: Build and demo instructions. Depends on: Working scripts. Done when: An unfamiliar engineer reproduces the complete demo.

.github/workflows/ci.yml

Implement: Run Solidity tests, fuzz target subset, TypeScript checks, Python tests, service integration and artifact builds. Interface: CI status on every pull request. Depends on: All package scripts. Done when: Broken contracts or generated schema drift block merge.

3. Shared schemas: packages/schemas

src/grant.ts

Implement: Zod or equivalent types: principal, agent identity, root/child grant, policy hash, validity, delegation depth, key epoch and revocation state. Interface: GrantInput, GrantView, DelegateRequest. Depends on: Chain ABI types. Done when: JSON and onchain codec round-trip without losing integer precision.

src/policy.ts

Implement: Define versioned access, financial, temporal, delegation and economic rules; forbid ambiguous default behavior. Interface: PolicyV1, PolicyRule, validatePolicy, canonicalPolicyHash. Depends on: Canonical encoding. Done when: Invalid token units, wildcard permissions and conflicting rules are rejected.

src/intent.ts

Implement: Canonical action types for transfer, escrow agreement and reviewed API call; bind chain, target, selector, payload digest and nonce. Interface: IntentV1, hashIntent. Depends on: EIP-712 domain config. Done when: Same input hashes identically across Node, browser, Python and Solidity.

src/exposure.ts

Implement: Use integer units to represent cashNow, periodSpend, grossLiability, requiredReservation, assumptions, adapter version and expiry. Interface: ExposureQuoteV1, quoteHash. Depends on: Action schemas. Done when: No floating point arithmetic in financial values.

src/decision.ts / src/approval.ts

Implement: Define DENY, REQUIRE_APPROVAL, ALLOW, typed reason codes and exact terms-bound approval payload. Interface: DecisionV1, ApprovalV1. Depends on: Grant/intent/exposure types. Done when: Stale or changed terms cannot reuse an approval.

src/obligation.ts / src/evidence.ts / src/receipt.ts

Implement: Define agreement terms, milestones, evidence metadata, verifier method, disputes, states and linked receipt references. Interface: TermsV1, EvidenceV1, ReceiptV1. Depends on: Canonical hashing. Done when: Public fields and private encrypted fields are clearly separated.

src/errors.ts / src/index.ts / schema/*.json

Implement: Export typed errors and generated JSON Schemas for Python and server validation. Interface: Single stable schema version. Depends on: All schema definitions. Done when: Schema drift test compares generated files to committed output.

4. Monad contracts: packages/contracts

foundry.toml / script/Deploy.s.sol / deployments/*.json

Implement: Pin compiler/toolchain; deploy contracts and publish addresses, chain ID and ABI checksum by network. Interface: forge build/test; deploy script. Depends on: Configured RPC and deployer. Done when: Repeatable testnet deployment with verifiable constructor settings.

src/interfaces/IPactTypes.sol

Implement: Structs/enums/events shared by contracts; no conflicting ABI versions. Interface: GrantInput, Authorization, ObligationState, events. Depends on: Shared schema mapping. Done when: ABI equivalence tests pass.

src/AuthorityRegistry.sol

Implement: Create signed root grants, attenuated children, key rotation, validity, ancestor revocation epochs and bounded delegation depth. Interface: createGrant, delegate, revoke, rotateKey, isActive. Depends on: Signature verifier; ledger allocation hook. Done when: Child never exceeds parent; parent revocation blocks new child actions.

src/CommitmentLedger.sol

Implement: Maintain per-grant per-asset ceiling, child allocation, pending reservation, active commitments and available capacity; ancestor checks atomic. Interface: allocateChild, reserve, commit, release, expireReservation, available. Depends on: AuthorityRegistry; role-safe callers. Done when: Concurrent proposals cannot oversubscribe a shared ceiling.

src/PactAccount.sol / src/PactAccountFactory.sol

Implement: Controlled wallet with supported action adapters; enforce active grant, exact call, nonce, expiry, approval and commitment reservation at execution. Interface: executeAuthorized, isValidSignature, account creation. Depends on: AuthorityRegistry, ledger, adapter registry. Done when: Unsupported arbitrary call, blanket approval and replay revert.

src/ActionAdapterRegistry.sol / src/adapters/ERC20TransferAdapter.sol

Implement: Allow only principal-authorized adapters; transfer adapter validates recipient/token/amount and exposes deterministic exposure bound. Interface: validateAndQuote, execute target. Depends on: PactAccount. Done when: A changed calldata payload fails authorization.

src/ObligationManager.sol

Implement: Consume reservation after both parties sign identical terms; store hash, funding requirements, verifier and state transitions. Interface: create, activate, submitEvidence, challenge, resolve, cancel. Depends on: Ledger, vault, signatures. Done when: Illegal transitions and expired party signatures revert.

src/SettlementVault.sol

Implement: Escrow deposit, milestone release, challenge hold and refund according to finalized obligation outcome. Interface: fund, settleMilestone, refund, balanceOfObligation. Depends on: ObligationManager. Done when: No double release, underfunded release or unauthorized withdrawal.

src/ReceiptRegistry.sol

Implement: Anchor receipt version and linked hashes for authorization, obligation, evidence, verdict and settlement. Interface: appendEvent, finalizeReceipt, getReceiptRoot. Depends on: ObligationManager / vault. Done when: No fabricated final settlement without valid upstream state.

src/libraries/TypedSignatures.sol

Implement: EIP-712 domain separation, nonces, deadlines, ERC-1271 smart account signature support and typed approval checks. Interface: verifyGrant, verifyApproval, verifyTerms. Depends on: Standard implementations. Done when: Cross-chain and cross-contract replay fail.

test/Authority.t.sol / test/LedgerInvariant.t.sol

Implement: Unit and stateful invariant tests over delegation, allocations, reservations, revocation and racing transactions. Interface: forge test and invariant harness. Depends on: Contract fixtures. Done when: All economic invariants hold through random action sequences.

test/Account.t.sol / test/Obligation.t.sol / test/Vault.t.sol

Implement: Execution denial, changed terms, funding, dispute and settlement tests using adversarial mocks. Interface: Comprehensive positive and negative cases. Depends on: Contracts and mock token/verifier. Done when: Failure path cannot leave inconsistent liability or funds.

5. Engine: packages/engine

src/normalizeIntent.ts

Implement: Validate and canonicalize incoming requests; never infer money amounts from free-form agent text. Interface: normalizeIntent(input): IntentV1. Depends on: Schemas. Done when: Unparseable or ambiguous action fails closed.

src/adapters/index.ts / transfer.ts / recurringAgreement.ts / apiCall.ts

Implement: Reviewed risk adapters with versioned upper-bound exposure; compute installment sums and termination penalties without double count. Interface: quote(intent, context): ExposureQuoteV1. Depends on: Schemas and pricing context. Done when: Unbounded liability returns UNSUPPORTED_ACTION.

src/simulate.ts

Implement: Use RPC call simulation for supported Monad actions, decode revert, detect expected effects and attach block reference. Interface: simulate(intent): SimulationResult. Depends on: Monad RPC and ABI. Done when: Simulation is labeled a snapshot, not proof of safety.

src/policyEvaluator.ts

Implement: Deterministic rule evaluation with DENY first, approval threshold second, ALLOW last; document inclusive threshold semantics. Interface: evaluate(grant, policy, intent, quote, state): DecisionV1. Depends on: Schemas, chain reads. Done when: Same fixtures produce same result in API and tests.

src/quoteService.ts / src/authorizationService.ts

Implement: Fetch active ancestry, quote, assess policy, reserve onchain and recheck state before operation submission. Interface: quote, authorize, reserveAuthority. Depends on: Adapter registry, policy engine, contract client. Done when: Capacity change cannot turn an old ALLOW into execution.

src/reconcile.ts

Implement: Handle tx receipt, provider response, failed send, timeout and duplicate request; never report settlement on a pending tx. Interface: reconcile(actionId). Depends on: Durable operation store. Done when: Every pending operation reaches confirmed, failed or recoverable state.

src/audit.ts

Implement: Emit correlation IDs and signed event digests with redaction and a coherent timeline. Interface: appendAuditEvent. Depends on: Structured log service. Done when: Every decision links to input hashes and policy version.

test/policy.test.ts / test/exposure.test.ts / test/race.test.ts

Implement: Cover thresholds, weird decimals, unsupported calls, changing price data, concurrent reservations and revocation. Interface: CI test suite. Depends on: Engine modules and chain fixtures. Done when: Risk cases match contract-level enforcement.

6. API service: apps/api

src/server.ts / src/config.ts

Implement: Initialize HTTP server, validated environment, DB, RPC, Engine and readiness endpoints. Interface: GET /health, GET /ready. Depends on: Engine, database. Done when: No traffic accepted before all critical dependencies are ready.

src/routes/grants.ts / policies.ts / approvals.ts

Implement: Authenticated grant creation, policy versioning and pending human approval; principal signature required for meaningful mutations. Interface: POST/GET grants, delegate, revoke, policies, approvals. Depends on: Auth, schemas, chain. Done when: Request auth alone never replaces principal signature.

src/routes/intents.ts / obligations.ts / receipts.ts

Implement: Quote/authorize/reserve, negotiate/sign terms, fund, submit evidence, challenge, settle, read receipt. Interface: Versioned /v1 routes. Depends on: Engine, contracts, object store. Done when: Response clearly distinguishes proposed, pending, confirmed and settled.

src/auth/principals.ts / auth/agents.ts

Implement: Verify human wallet/passkey sessions and agent proof-of-possession credentials with scoped tenant resolution. Interface: requirePrincipal, requireAgent. Depends on: Session store, signatures. Done when: Agent cannot act for another tenant by changing request IDs.

src/db/schema.sql / db/migrations/*

Implement: Projection tables for policies, encrypted evidence metadata, approval work queue, idempotency and operations. Interface: Migration commands. Depends on: PostgreSQL. Done when: Fresh database migrates and restores projections from chain events.

src/openapi.ts / openapi.yaml

Implement: Document every request/response schema, auth method and error code. Interface: OpenAPI v3. Depends on: Shared schemas. Done when: SDK generation checks against committed API specification.

7. API Gateway and vault: apps/gateway

src/proxy.ts

Implement: Single outbound path for allowlisted providers; validate host, method, endpoint, content type, payload digest and redirect. Interface: POST /v1/proxy/:provider/:operation. Depends on: Agent auth, engine. Done when: Unauthorized method or redirected destination never receives secret.

src/secrets.ts

Implement: Encrypt stored provider credential, obtain short-lived decrypt permission, rotate and revoke; never log or return plaintext. Interface: readProviderSecret after ALLOW. Depends on: KMS/vault service. Done when: Agent API, logs and MCP output contain no provider secret.

src/adapters/providerA.ts

Implement: One real provider integration with typed endpoints, maximum usage/cost calculation and response parser. Interface: quoteProviderCall, forward, reconcile. Depends on: Proxy and schemas. Done when: Unbounded provider action denied before forwarding.

src/idempotency.ts / src/usage.ts

Implement: Persist idempotency keys and reserve maximum charge, reconcile actual provider usage, release unused amount. Interface: reserveUsage, finalizeUsage. Depends on: DB, Engine ledger. Done when: Retries cannot double execute or double charge policy capacity.

test/proxy.test.ts / test/ssrf.test.ts

Implement: Check leaked credentials, redirects, DNS rebinding, host normalization and forbidden payload fields. Interface: Gateway security suite. Depends on: Mock provider. Done when: All bypass cases fail closed.

8. SDKs and proof verification

packages/sdk-ts/src/client.ts / grants.ts / obligations.ts

Implement: Typed PactClient with quote, authorize, reserve, verify, delegate, revoke, create agreement, evidence and receipt calls. Interface: Async methods with exact return states. Depends on: OpenAPI and schemas. Done when: Example agent operates without manual HTTP handling.

packages/sdk-ts/src/signing.ts / errors.ts

Implement: Canonical EIP-712 signing and typed error decoding, nonces, chain checks, abort and retry policy. Interface: signGrant, signTerms, PactError. Depends on: Wallet client and schemas. Done when: Signature fixtures match Solidity.

packages/sdk-python/pact/client.py / models.py

Implement: Python parity for core flows with strict integer amounts and generated schemas. Interface: PactClient and typed Pydantic models. Depends on: OpenAPI/schema output. Done when: TS/Python cross-language hash fixtures match.

packages/sdk-python/pact/signing.py / errors.py

Implement: EIP-712 encoding, nonces and chain/domain checks; no principal key baked into examples. Interface: Signature helpers and typed exceptions. Depends on: Python EVM library. Done when: Replay and mismatched-chain tests fail.

packages/proofs/src/verifyAuthority.ts

Implement: Independently query Monad state, confirm grant ancestry, action hash, live reservation, liability, expiry and terms binding. Interface: verifyAuthority(proof, rpc). Depends on: Contract ABIs. Done when: Provider verifies proof with Pact API turned off.

packages/proofs/src/verifyReceipt.ts

Implement: Reconstruct receipt links, contract events, disclosure digest and labels for external attestations. Interface: verifyEconomicReceipt(bundle, rpc). Depends on: Indexer optional, Monad mandatory. Done when: Tampered evidence or missing settlement is detected.

9. MCP server: apps/mcp

src/server.ts / src/auth.ts

Implement: Expose authenticated MCP server; bind session to principal, agent and grant; require scoped credentials. Interface: MCP transport and identity context. Depends on: SDK, API auth. Done when: Prompt text cannot choose another principal.

src/tools/read.ts

Implement: Implement pact_get_my_authority, pact_quote_action, pact_get_obligation and pact_get_receipt with disclosure limits. Interface: Typed read tool definitions. Depends on: SDK, schemas. Done when: No cross-tenant policies or private evidence revealed.

src/tools/write.ts

Implement: Implement pact_propose_obligation, pact_request_approval, pact_execute_approved_action, pact_submit_evidence. Interface: Typed write tools, same Engine path. Depends on: SDK, approvals. Done when: MCP call cannot bypass principal approval or account guard.

test/tools.test.ts / test/injection.test.ts

Implement: Validate tool schemas, resource boundaries, prompt injection in retrieved evidence and write denial. Interface: MCP integration tests. Depends on: Mock client. Done when: Untrusted tool content never changes authorization.

10. Indexer, Console, Explorer

apps/indexer/src/stream.ts / handlers.ts / reorg.ts

Implement: Ingest Monad events into rebuildable views; maintain finality marker, cursor and rollback logic. Interface: Event handlers for all contracts. Depends on: ABIs, database. Done when: Replay from genesis reconstructs same projection.

apps/console/app/(app)/agents/page.tsx / components/GrantForm.tsx

Implement: Agent creation and key binding, root and child grant editor with hard preview of capacity and duration. Interface: Human readable signable summary. Depends on: API and wallet auth. Done when: Principal sees exact liability ceiling before signing.

apps/console/app/(app)/policies/page.tsx / approvals/page.tsx

Implement: Policy diff/version editor, deterministic dry-run, approval queue with exact intent, terms, cash now and gross liability. Interface: PublishPolicy and ApproveIntent flows. Depends on: API, engine preview. Done when: Changed terms require new approval.

apps/console/app/(app)/obligations/page.tsx / audit/page.tsx

Implement: Milestone, verifier, dispute and settlement timeline; link each event to proof and Monad transaction. Interface: Obligation detail and audit timeline. Depends on: Indexer/read API. Done when: Cannot display pending transaction as finalized.

apps/explorer/app/proof/[id]/page.tsx / receipt/[id]/page.tsx

Implement: Public scoped proof verifier, status and disclosed terms; verification method and evidence trust label. Interface: Verify button using proof package. Depends on: Public RPC and proof bundle. Done when: Still verifies when Pact API unavailable.

apps/console/components/CapacityLedger.tsx / ExposureBreakdown.tsx

Implement: Explain total, allocated, reserved, committed, available, immediate spend and escrow separately. Interface: Accessible ledger components. Depends on: Read API. Done when: 6,000 future liability never appears as 1,000 spend only.

11. Infrastructure, tests and docs

infra/docker/compose.yml

Implement: Local Postgres, encrypted object storage emulator, optional secrets emulator, services and health checks. Interface: docker compose up. Depends on: Service Dockerfiles. Done when: Cold start reaches healthy state reproducibly.

infra/deploy/*.yaml / monitoring/alerts.yaml

Implement: Separate API, Gateway, MCP, indexer deployments; secret injection, TLS, logging, metrics and alarms. Interface: Deployment manifests. Depends on: Target hosting provider. Done when: Gateway credentials never embedded in images or client bundles.

tests/invariants/*.t.sol

Implement: Stateful action sequences over grants, delegation, reserve/commit/release, escrow and revocation. Interface: Foundry invariant suite. Depends on: Contracts. Done when: No capacity or funding invariant violation.

tests/integration/lifecycle.test.ts

Implement: Real contract plus service path for approval, provider proof check, agreement, evidence, settlement and receipt. Interface: One end-to-end lifecycle. Depends on: Local chain and services. Done when: Final receipt independently reconstructs all states.

tests/e2e/procurement.spec.ts

Implement: Browser demo: deny forbidden endpoint, deny over-limit transfer, deny future liability, approve revised valid agreement. Interface: Playwright scenario. Depends on: Deployed demo services. Done when: Judges can run and observe the full chain.

docs/protocol/ACCOUNTING.md / PROOFS.md / THREAT_MODEL.md

Implement: Mathematical accounting, proof field definitions, security and trust boundaries with examples. Interface: Protocol reference. Depends on: Implementation. Done when: Docs match executable behavior.

docs/api/QUICKSTART.md / operations/RUNBOOK.md

Implement: SDK demo, testnet deployment, credential rotation, stuck operation recovery, index rebuild and incident steps. Interface: Operator and builder guides. Depends on: Stable services. Done when: A second engineer can deploy and recover the system.

12. Build sequence for Codex

Phase	Files to finish	Gate
A Schema + tests	schemas, Foundry fixtures, shared signature vectors	Typed data hash parity across Solidity, TS and Python.
B Authority and account	AuthorityRegistry, Ledger, PactAccount, adapters	Revocation, attenuation, racing reservation and bypass tests.
C Economic lifecycle	ObligationManager, Vault, ReceiptRegistry	Funded agreement with valid challenge and settlement.
D Engine and API	Adapters, policy evaluator, authorization service, API	Full deny/approval/allow on real Monad calls.
E Gateway and SDKs	Provider adapter, secrets vault, TS/Python SDK, proof package	No direct credentials; independent verification works.
F Product surfaces	MCP, Console, Explorer, indexer, example agent	Demo runs end to end with traceable evidence.
G Hardening	Invariant fuzzing, threat review, monitoring and runbook	No critical findings and reproducible deployment.

Definition of done

The first fully working slice must include signed principal grant, attenuated child, blocked over-limit action, approval for a valid commitment, atomic reservation, counterparty proof verification, signed obligation, escrowed milestone, submitted evidence, specified verifier result, dispute/timeout path, settlement and independently verified economic receipt. All other UI and integrations are secondary to this chain.

Explicit unresolved deployment choices: actual Monad network, supported stablecoin address and decimals, external provider for Gateway demo, verifier model, object storage provider, operational signing model, dispute arbitrator and upgrade policy. Codex should validate these and record choices in an ADR before handling real funds. Sponsor bounty integrations require the individual bounty rules.